

WorkCore API Integration Guide

Version: 1.7

Date: March 17, 2026

Default public API URL: `https://api.hq21.tech`

Partner-specific host policy may pin canonical host (for `partner_id=epam_future-insurance`): `https://api.runwcr.com`).

1. Purpose

This document is a practical integration guide for backend and platform teams that need to integrate with the WorkCore runtime API.

Source-of-truth contract:

- OpenAPI: `https://api.hq21.tech/openapi.yaml`
- API reference: `https://api.hq21.tech/api-reference`
- Agent integration endpoint: `https://api.hq21.tech/agent-integration-kit`

Canonical onboarding surfaces:

- `GET /integration-capabilities` (public machine-readable capability negotiation contract)
- `GET /agent-integration-kit` (human-readable guide)
- `GET /agent-integration-kit.json` (machine-readable bundle with canonical `integration_manifest`)
- `GET /agent-integration-test` (doctor UI)
- `GET /agent-integration-test.json` (doctor JSON report)
- `POST /agent-integration-test/validate-draft` (draft lint/validation)
- `GET /internal/partner-access` (internal operator onboarding portal)
- `POST /internal/partner-access/onboard-package` (download canonical onboarding ZIP bundle)

Gateway host policy:

- Canonical host is declared by `integration_manifest.host_policy`.
- `host_policy.mode=request_host` means use the public host returned by onboarding/doctor surfaces.
- `host_policy.mode=pinned` means integrations must use `host_policy.canonical_base_url` and `host_policy.allowed_domains`.
- For `partner_id=epam_future-insurance`, policy is pinned to `https://api.runwcr.com`.

Current conformance baseline for consuming services:

- canonical chat path is `POST /chat`
- typed platform errors are parsed and respected
- capabilities are negotiated via `GET /integration-capabilities`

- integration readiness is gated by `GET /agent-integration-test.json`
- canonical host selection is taken from `integration_manifest.host_policy`
- direct orchestrator routing is verified through `POST/GET /projects/{project_id}/workflow-definitions*`

2. Authentication model

WorkCore API is protected with OAuth2 access tokens issued by Microsoft Entra ID (`client_credentials` flow).

Token exchange:

- Token endpoint:
 - `https://login.microsoftonline.com/<tenant_id>/oauth2/v2.0/token`
- Form fields:
 - `grant_type=client_credentials`
 - `client_id=<partner_client_id>`
 - `client_secret=<partner_client_secret>`
 - `scope=api://workcore-partner-api/.default`
- Use returned token as:
 - `Authorization: Bearer <access_token>`
- JWT note:
 - when decoding the token, `aud` may appear as the WorkCore resource app ID instead of the scope alias; this is expected if the token was issued for the scope above

No OAuth token is required for:

- `GET /health`
- `GET /openapi.yaml`
- `GET /api-reference`
- `GET /workflow-authoring-guide`
- `GET /agent-integration-kit`
- `GET /agent-integration-kit.json`
- `GET /agent-integration-test`
- `GET /agent-integration-test.json`
- `POST /agent-integration-test/validate-draft`
- `GET /schemas/*`
- `POST /webhooks/inbound/{integration_key}` (signature-based)

Bearer auth is required for:

- `GET /agent-integration-logs`

3. Required headers

Common headers

- `Authorization: Bearer <token>` (required on protected endpoints)
- `X-Tenant-Id: <tenant>` (recommended on all calls, required for strict multi-tenant paths such as `/chat`)
- `X-Correlation-Id: <id>` (recommended)
- `X-Trace-Id: <id>` (recommended)

Workflow authoring/run headers

- `X-Project-Id: <project_id>` is required for `/workflows*` APIs.

Retry safety

- `Idempotency-Key: <key>` is strongly recommended for mutating requests (`POST/PATCH/DELETE`).

4. Quick start

```
export BASE_URL="https://api.hq21.tech"
# partner-specific pinned policy example:
# export BASE_URL="https://api.runwcr.com" # required for partner_id=epam_future-
insurance
export ENTRA_TENANT_ID="<entra_tenant_id>"
export CLIENT_ID="<partner_client_id>"
export CLIENT_SECRET="<partner_client_secret>"
export SCOPE="api://workcore-partner-api/.default"
export TENANT_ID="local"
export PROJECT_ID="proj_hq21"

TOKEN="$(curl -sS -X POST
"https://login.microsoftonline.com/${ENTRA_TENANT_ID}/oauth2/v2.0/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d
"grant_type=client_credentials&client_id=${CLIENT_ID}&client_secret=${CLIENT_SECRET}
"&scope=${SCOPE}" \
| jq -r '.access_token')"
```

Fetch canonical integration manifest:

```
curl -sS "$BASE_URL/agent-integration-kit.json" | jq '.integration_manifest'
```

Fetch platform negotiation contract:

```
curl -sS "$BASE_URL/integration-capabilities" | jq
```

Run integration doctor:

```
curl -sS "$BASE_URL/agent-integration-test.json" | jq '.summary, .checks[] | {id,
status, code}'
```

Health check:

```
curl -sS "$BASE_URL/health"  
# expected: ok
```

List projects (protected endpoint):

```
curl -sS "$BASE_URL/projects" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID"
```

List workflows (requires project scope):

```
curl -sS "$BASE_URL/workflows" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "X-Project-Id: $PROJECT_ID"
```

5. Minimal integration flow

Contract separation:

- `/capabilities*` is the versioned capability contract registry.
- `GET /integration-capabilities` is the external-client negotiation surface.

5.1 Create project

```
curl -sS -X POST "$BASE_URL/projects" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "Content-Type: application/json" \  
-d '{  
  "project_id": "proj_hq21",  
  "project_name": "HQ21 Production"  
}
```

5.2 Create workflow draft

```
curl -sS -X POST "$BASE_URL/workflows" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "X-Project-Id: $PROJECT_ID" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Order triage",  
  "description": "Classify customer intent and route",  
  "draft": {
```

```

    "nodes": [
      {"id": "start", "type": "start"},
      {"id": "end", "type": "end"}
    ],
    "edges": [
      {"source": "start", "target": "end"}
    ]
  }
}'

```

5.3 Publish workflow

```

curl -sS -X POST "$BASE_URL/workflows/<workflow_id>/publish" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Tenant-Id: $TENANT_ID" \
-H "X-Project-Id: $PROJECT_ID"

```

5.4 Register workflow in project routing index (required for direct orchestrator mode)

```

curl -sS -X POST "$BASE_URL/projects/$PROJECT_ID/workflow-definitions" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Tenant-Id: $TENANT_ID" \
-H "Content-Type: application/json" \
-d '{
  "workflow_id": "<workflow_id>",
  "name": "Order triage",
  "description": "Classify customer intent and route",
  "tags": ["customer-support", "triage"]
}'

```

5.5 Read back routing registration (required before direct-routing smoke/cutover)

```

curl -sS "$BASE_URL/projects/$PROJECT_ID/workflow-definitions/<workflow_id>" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Tenant-Id: $TENANT_ID"

```

Routing note:

- If your service uses project-scoped chat only (`POST /chat`) and does not rely on direct orchestrator routing, treat sections 5.4 and 5.5 as not applicable.
- If your service calls `POST /orchestrator/messages` in direct-routing mode, the readback step is mandatory before smoke/cutover.

5.6 Configure project default chat workflow

```
curl -sS -X PATCH "$BASE_URL/projects/$PROJECT_ID" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "settings": {
      "default_chat_workflow_id": "<workflow_id>"
    }
  }'
```

5.7 Start run directly (non-chat path)

```
curl -sS -X POST "$BASE_URL/workflows/<workflow_id>/runs" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Idempotency-Key: run-$(date +%s)" \
  -H "Content-Type: application/json" \
  -d '{
    "inputs": {
      "customer_text": "I need help with my order"
    }
  }'
```

5.8 Poll run state

```
curl -sS "$BASE_URL/runs/<run_id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID"
```

Diagnostic fields in failed runs:

- `error` / `last_error` (top-level message)
- `failed_node_id` (top-level node source)
- `node_runs[]` (canonical node details)
- `node_states[]` (backward-compatible alias of `node_runs[]`)

5.9 Diagnose failed runs (mandatory for incidents)

1. Inspect run node diagnostics:

```
curl -sS "$BASE_URL/runs/<run_id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
```

```
-H "X-Project-Id: $PROJECT_ID" \  
| jq '.node_runs[] | {node_id, status, last_error}'
```

2. Inspect immutable ledger trail:

```
curl -sS "$BASE_URL/runs/<run_id>/ledger?limit=200" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "X-Project-Id: $PROJECT_ID" \  
| jq '.items[] | {timestamp, event_type, status, step_id, error: .payload.error}'
```

Ledger compatibility note:

- each entry includes both `step_id` and `node_id` (same value for node-scoped events).

6. SSE streaming and reconnect

Run progress stream:

```
curl -N "$BASE_URL/runs/<run_id>/stream" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "X-Project-Id: $PROJECT_ID" \  
-H "Accept: text/event-stream"
```

Reconnect from last event:

```
curl -N "$BASE_URL/runs/<run_id>/stream" \  
-H "Authorization: Bearer $TOKEN" \  
-H "X-Tenant-Id: $TENANT_ID" \  
-H "X-Project-Id: $PROJECT_ID" \  
-H "Last-Event-ID: evt_abc123"
```

SSE events include `id`, `event`, and JSON `data` with:

- `event_id`, `sequence`, `timestamp`
- `run_id`, `workflow_id`, `version_id`, `node_id`
- `type`, `payload`
- `correlation_id`, `trace_id`, `tenant_id`, `project_id`, `import_run_id`

7. Webhooks

7.1 Inbound webhook (to WorkCore)

Endpoint:

- `POST /webhooks/inbound/{integration_key}`

Headers:

- `X-Webhook-Timestamp: <unix_ts>`

- `X-Webhook-Signature: t=<unix_ts>,v1=<hmac_sha256>`
- optional `Idempotency-Key: <key>`

Signature payload:

- `"<timestamp>.<raw_request_body>"`

Signature example:

```
SECRET="<inbound_secret>"
TS="$(date +%s)"
BODY='{"action": "start_run", "workflow_id": "wf_123", "inputs":
{"customer_text": "Hello"}}'
SIG="$(printf "%s.%s" "$TS" "$BODY" | openssl dgst -sha256 -hmac "$SECRET" -hex |
sed 's/^.* //')"

curl -sS -X POST "$BASE_URL/webhooks/inbound/default" \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Timestamp: $TS" \
  -H "X-Webhook-Signature: t=$TS,v1=$SIG" \
  -H "Idempotency-Key: wh-$(date +%s)" \
  -d "$BODY"
```

Supported actions:

- `start_run`
- `resume_interrupt`

7.2 Outbound webhooks (from WorkCore)

Register subscription:

```
curl -sS -X POST "$BASE_URL/webhooks/outbound" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://your-service.example.com/workcore/events",
    "event_types": ["run_completed", "run_failed", "interrupt_created",
"node_failed"]
  }'
```

List subscriptions:

```
curl -sS "$BASE_URL/webhooks/outbound" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID"
```

Delete subscription:


```
curl -sS -X DELETE "$BASE_URL/webhooks/outbound/<subscription_id>" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Tenant-Id: $TENANT_ID"
```

8. Chat endpoint

Chat endpoint is on the same API host:

- `POST /chat`
- `POST /chatkit` (deprecated compatibility alias during transition window)

Requirements:

- `X-Tenant-Id` is required
- send `Authorization: Bearer <access_token>` obtained from Entra OAuth token endpoint
- when using deprecated alias `POST /chatkit` before sunset, expect headers:
 - `Deprecation: true`
 - `Sunset: Sat, 04 Apr 2026 00:00:00 GMT`

For `threads.create`, resolution order is:

- `metadata.workflow_id` -> explicit workflow mode (backward-compatible)
- else `metadata.project_id` -> resolve `projects.settings.default_chat_workflow_id`
- else `X-Project-Id` -> resolve `projects.settings.default_chat_workflow_id`
- else `CHAT_PROJECT_SCOPE_REQUIRED`

Canonical onboarding manifest expectations:

- `integration_manifest.chat_api_url` must always point to `/chat`
- `integration_manifest.integration_capabilities_url` must point to `GET /integration-capabilities`
- `integration_manifest.deprecated_chat_alias_url` may include `/chatkit` only as deprecated compatibility alias
- `integration_manifest.host_policy` is first-class and defines canonical host behavior per partner:
 - `mode=request_host` -> use `api_base_url` from onboarding surfaces
 - `mode=pinned` -> use only `canonical_base_url` and `allowed_domains` from policy
- if `/chatkit` appears in partner config/docs:
 - before `2026-04-04T00:00:00Z` => doctor emits `WARN`
 - on/after `2026-04-04T00:00:00Z` => doctor emits `FAIL`

Project-scoped chat defaults require a configured published workflow:

- missing project setting -> `CHAT_DEFAULT_WORKFLOW_NOT_CONFIGURED`
- configured workflow missing/inactive/unpublished -> `CHAT_DEFAULT_WORKFLOW_NOT_FOUND`

Diagnostic checks:

```
# expected: 200 + SSE events
curl -N -X POST "$BASE_URL/chat" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Content-Type: application/json" \
  -d '{
    "type":"threads.create",
    "metadata":{"project_id":"'$PROJECT_ID'",},
    "params":{"input":{"content":
[{"type":"input_text","text":"start"}],"attachments":[],"inference_options":{}}}
  }'

# compatibility alias (before 2026-04-04T00:00:00Z): same behavior as /chat +
deprecation headers
curl -i -X POST "$BASE_URL/chatkit" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Content-Type: application/json" \
  -d '{"type":"threads.create","metadata":
{"project_id":"'$PROJECT_ID'",,"params":{"input":{"content":
[{"type":"input_text","text":"start"}],"attachments":[],"inference_options":{}}}}}'

# starting 2026-04-04T00:00:00Z: expected 410 Gone
curl -i -X POST "$BASE_URL/chatkit" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant-Id: $TENANT_ID" \
  -H "X-Project-Id: $PROJECT_ID" \
  -H "Content-Type: application/json" \
  -d '{"type":"threads.create","metadata":
{"project_id":"'$PROJECT_ID'",,"params":{"input":{"content":
[{"type":"input_text","text":"start"}],"attachments":[],"inference_options":{}}}}}'
```

9. Error model

All integration HTTP failures use additive typed `PlatformErrorEnvelope`:

```
{
  "error": {
    "code": "INVALID_ARGUMENT",
    "message": "...",
    "category": "validation",
    "retryable": false,
    "retry_after_s": null,
    "bad_fields": null,
```

```

    "unsupported_feature": null,
    "docs_ref": null,
    "details": null,
    "correlation_id": "corr_123"
  },
  "correlation_id": "corr_123"
}

```

Backward compatibility:

- existing parsers of `error.code`, `error.message`, and top-level `correlation_id` are unchanged.
- additional typed fields are additive and may be `null`.

Retry behavior:

- when `error.retryable=true`, client can safely retry.
- when backoff is known, API sets `error.retry_after_s` and may emit HTTP `Retry-After` (notably on `429` / `503`).

Category values:

- `auth`, `validation`, `configuration`, `not_found`, `conflict`, `unsupported_feature`, `transient`, `internal`, `route`, `action`.

Common integration errors:

- `401 UNAUTHORIZED`: missing, expired, or invalid OAuth access token
- `404 ERR_PROJECT_NOT_FOUND`: project scope does not exist in the tenant
- `422 ERR_PROJECT_ID_REQUIRED`: missing `X-Project-Id` on workflow endpoints
- `422 CHAT_PROJECT_SCOPE_REQUIRED`: `threads.create` omitted both `metadata.workflow_id` and project scope
- `409 CHAT_DEFAULT_WORKFLOW_NOT_CONFIGURED`: project exists but has no `settings.default_chat_workflow_id`
- `404 CHAT_DEFAULT_WORKFLOW_NOT_FOUND`: configured default chat workflow is missing, inactive, or unpublished
- `400 BadRequest` with Azure OpenAI text:
 - `"Responses API is enabled only for api-version 2025-03-01-preview and later"`
 - Fix: set `AZURE_OPENAI_API_VERSION=2025-03-01-preview` (or newer) consistently in Key Vault and runtime environment.
- `400 INVALID_ARGUMENT`: invalid request body/parameters

10. Production checklist for external teams

1. OAuth client secret rotation process is defined and tested (12-month lifetime + overlap window).
2. Client reads `GET /integration-capabilities` and does not hardcode capability assumptions from prose docs only.

3. Latest `GET /agent-integration-test.json` verdict is `PASS` or an explicitly accepted `WARN` with owner/remediation.
4. `integration_manifest.host_policy` is enforced for canonical base URL and allowed domains.
5. `X-Tenant-Id`, `X-Correlation-Id`, `X-Trace-Id` are generated and propagated end-to-end.
6. `X-Project-Id` is always set for `/workflows*` APIs.
7. Typed platform errors are parsed; retries happen only when `error.retryable=true` and respect `error.retry_after_s` / `Retry-After`.
8. All mutating calls send stable `Idempotency-Key`.
9. SSE consumers support reconnect with `Last-Event-ID`.
10. Webhook consumers verify signatures and handle retries idempotently.
11. Direct orchestrator routing integrations confirm `GET /projects/{project_id}/workflow-definitions/{workflow_id}` after bind and before smoke/cutover.
12. Monitoring dashboards alert on 401/429/5xx spikes and unexpected `/chatkit` traffic.

11. Operational contacts

- API contract and behavior changes must be validated against `openapi.yaml`.
- Invite-only onboarding package for partners:
 - `docs/integration/apim-partner-onboarding-guide.md`
- For environment-specific token/domain issues, contact the WorkCore platform owner.